Extended reading — Week 13: Multi-Agent RL and AlphaZero

October 6, 2026

Five papers

1. Silver et al. — *Mastering the game of Go without human knowledge* (Nature 2017)

AlphaGo Zero. The paper that replaced AlphaGo's expert-game-bootstrapped training with pure self-play and a single policy-value network. The algorithmic spine of this week's mini-project.

Why it matters. Before AGZ, "reinforcement learning" meant "train against an existing baseline". AGZ showed that tabula-rasa self-play could reach super-human play in three days on TPUs — no human games, no hand-crafted features. The training loop is exactly the self-play + MCTS + policy/value loop you implemented in the project.

> "Starting from random play, and given no domain knowledge except the game rules, AlphaGo Zero achieved superhuman performance, winning 100-0 against the previously published, champion-defeating AlphaGo."

Commentary. The 100-0 is striking but the algorithmic content is the more important contribution: a unified policy/value network, MCTS that uses both heads, and a self-improvement training loop with no external supervision. Every reproduction of AlphaGo Zero (and AlphaZero, KataGo, MuZero) follows the same template.

2. Silver et al. — *A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play* (Science 2018)

AlphaZero. The generalisation of AGZ to chess and shogi: same algorithm, no game-specific code, no opening books, no endgame tablebases. Beat Stockfish (the world's strongest chess engine) and Elmo (the strongest shogi engine).

Why it matters. It proved that the AGZ recipe was *general* — it works for any two-player zero-sum game with perfect information. The chess result was particularly striking: chess had been "solved" by classical alpha-beta search for two decades; AlphaZero's neural-MCTS beat the best alpha-beta engine after 4 hours of training.

> "The same general-purpose reinforcement learning algorithm achieves superhuman performance in chess, shogi, and Go."

Commentary. The key technical change from AGZ to AZ was the removal of the game-symmetry augmentation (Go has 8-fold symmetry, chess has none). Otherwise the architecture is identical. The lesson: the algorithm itself does the heavy lifting; domain knowledge enters via the environment, not via custom training tricks.

3. Yu et al. — *The Surprising Effectiveness of PPO in Cooperative Multi-Agent Games* (NeurIPS 2022, arXiv:2103.01955)

MAPPO. The paper that established centralised-PPO as the production default for cooperative MARL, displacing MADDPG / QMIX / COMA on most benchmarks despite being algorithmically simpler.

Why it matters. MAPPO is what you implemented in the project. It is the right default. The paper's ablations identified the key tricks: careful value normalisation, agent-specific value targets, large minibatches. The lesson: simpler algorithms with careful engineering often beat more sophisticated algorithms with naive engineering.

> "With proper implementation tricks — value normalisation, agent-specific features in the centralised critic, action masking, and large minibatch size — MAPPO matches or exceeds SOTA across SMAC, MA-MuJoCo, and Hanabi."

Commentary. The implementation-trick list is invaluable; the paper's appendix has the cleanest description of MAPPO best practice anywhere. Use it as a checklist when scaling MAPPO to a new env.

4. Rashid et al. — *QMIX: Monotonic Value Function Factorisation for Deep Multi-Agent Reinforcement Learning* (ICML 2018)

QMIX. Value-decomposition for cooperative MARL: factorise the joint Q-function as a monotone aggregator over per-agent Q-functions; this allows decentralised greedy execution that is consistent with joint argmax.

Why it matters. For value-based MARL methods, QMIX is the SOTA baseline. Its monotonicity restriction is also the canonical example of the trade-off between expressiveness and decomposability — QMIX is decentralisable but cannot represent XOR-style coordination.

> "We enforce that Q_{tot} is monotonic in each agent's Q_a, which ensures that the joint argmax operation on Q_{tot} can be performed by selecting each agent's individual argmax of Q_a."

Commentary. Variants like QPLEX (Wang et al. ICLR 2021) relax the monotonicity to a richer dueling decomposition; the QPLEX paper also documents the XOR counter-example clearly. Read QMIX first then QPLEX to understand the full design space.

5. Lanctot et al. — *A Unified Game-Theoretic Approach to Multiagent Reinforcement Learning* (NeurIPS 2017, arXiv:1711.00832)

PSRO. The framework that unifies self-play, fictitious play, double-oracle, and replicator dynamics into a single iterated-best-response algorithm.

Why it matters. PSRO is what you should use for general-sum MARL (or for cyclic-dynamics zero-sum games like Rock-Paper-Scissors generalisations). The meta-game is solved by classical game theory; the inner "best response" step is single-agent RL. The decoupling is the algorithmic insight.

> "We propose Policy-Space Response Oracles (PSRO), an extension of double-oracle algorithms to multi-agent RL settings. PSRO unifies fictitious play (with uniform meta-game solver), self-play (with deterministic meta-game solver), and double-oracle (with Nash meta-game solver) into a single algorithmic framework."

Commentary. The 2019 AlphaStar work uses a PSRO-derived "league training" approach: the meta-game solver is the league-prioritised sampling distribution. PSRO is the right framework for any non-transitive-game RL deployment.

Bonus paper: Vinyals et al. — *Grandmaster level in StarCraft II using multi-agent reinforcement learning* (Nature 2019)

AlphaStar. The most ambitious public MARL system to date. League training with main agents, main exploiters, league exploiters; supervised pre-training from human games; large-scale distributed self-play.

Why it matters. Pulls together everything in this week: CTDE (actors take local observations, critics take the global state), self-play (at the league level), and best-response training (per main agent). The Nature paper has the cleanest published account of league training.

> "Without the additional exploiters in the league, the main agents collapse onto rock-paper-scissors strategies. The diversity of the league is the key to robust play."

Commentary. The collapse-onto-RPS-cycles finding is the canonical example of why naive self-play fails in non-transitive games. The fix — explicit exploiters — is structural, not algorithmic. Any production multi-agent RL system on real-world economic games will need a similar structural fix.

Bonus paper: Schrittwieser et al. — *Mastering Atari, Go, chess and shogi by planning with a learned model* (Nature 2020)

MuZero. Extends AlphaZero to environments where the dynamics are unknown: the agent learns a latent dynamics model alongside the policy and value, and does MCTS in the latent space.

Why it matters. It is the single algorithm that handles both board games (perfect-info, known transitions) and Atari (pixel inputs, unknown transitions). MuZero set a new SOTA on Atari while matching AlphaZero on the board games.

> "MuZero matches the performance of AlphaZero in Go, chess and shogi, and improves upon the state of the art in Atari without any prior knowledge of the game rules."

Commentary. The latent-dynamics model is trained end-to-end with policy and value losses. EfficientZero (Ye et al. 2021) shows that a self-supervised consistency loss between predicted and actual latents accelerates this further; the data efficiency improvements are dramatic ($\sim 100\times$ less data for comparable Atari scores).

Glossary

CTDE. Centralised Training, Decentralised Execution. Critic sees all; actor sees local observation only.

PUCT. Predictor + UCB applied to Trees; the AlphaZero selection rule.

Dirichlet noise. Exploration noise added to the prior at the MCTS root only. Essential to prevent policy collapse.

Symmetry augmentation. For games with board symmetries, each training tuple is augmented with its 8 (or 4, or 2) symmetric versions.

Self-play. Both agents in a 2-player game share the same policy network; the network plays itself.

Fictitious play. Each agent best-responds to the empirical distribution of historical opponent actions.

PSRO. Policy-Space Response Oracles; meta-game over a policy population.

League training. AlphaStar's PSRO variant with main agents and explicit exploiter populations.

QMIX monotonicity. The aggregator from per-agent Qs to joint Q is monotone, which makes decentralised greedy consistent with joint argmax.

MuZero. AlphaZero with a learned latent dynamics model.

Historical context: how we got here

The thread from Samuel's checkers program (1959) to AlphaZero is a 60-year story of intelligent search getting steadily better priors. The milestones:

1959 — Samuel's checkers. Self-play with hand-crafted features; the first ML system to demonstrate genuine improvement through play. Samuel's machine learned its evaluation function via TD-like updates on tens of thousands of self-play games.

1992 — TD-Gammon (Tesauro). TD(λ) on a neural-net backgammon evaluator. World-class play on a single workstation; first demonstration that neural nets + TD + self-play could match expert human play in a complex domain.

1997 — Deep Blue beats Kasparov. Hand-crafted features + massive alpha-beta search. The end of the "classical" chess era.

2007 — MCTS (UCT). Kocsis & Szepesvári; the algorithm that gave bandit-style exploration to game tree search. Powered the first super-human computer Go programs (CrazyStone, Zen).

2013 — DQN (Mnih et al.). Deep RL on Atari. Proved that neural nets + replay buffer + target network could learn pixel-to-action mappings.

2016 — AlphaGo. MCTS + policy network trained on human games + value network. Beat Lee Sedol 4-1.

2017 — AlphaGo Zero. Removed the human-game bootstrap; pure self-play. The algorithm of this week's mini-project.

2018 — AlphaZero. Generalised AGZ to chess, shogi, and Go with a single algorithm. The proof that the recipe is domain-agnostic.

2019 — AlphaStar. League training for non-transitive games. StarCraft II at grandmaster level.

2020 — MuZero. Latent dynamics model; AlphaZero without knowing the rules. Atari-level play with the same algorithm that masters Go.

2022 — MAPPO surprise. The simplest CTDE PPO matches or beats every more-sophisticated MARL algorithm on the standard benchmarks. Lesson: implementation engineering matters more than algorithmic sophistication.

Annotated reading order

Weekend 1 (foundations). Silver et al. Nature 2017 (AlphaGo Zero) end-to-end, then Silver et al. Science 2018 (AlphaZero). Both papers are short; the supplementary materials are where the implementation details live — read those.

Weekend 2 (cooperative MARL). Yu et al. NeurIPS 2022 (MAPPO); Rashid et al. ICML 2018 (QMIX); Lowe et al. NeurIPS 2017 (MADDPG). Together these give you the three flavours of CTDE: PPO-style on-policy, DQN-style value-decomposition, DDPG-style off-policy.

Weekend 3 (competitive / general-sum). Lanctot et al. NeurIPS 2017 (PSRO); Vinyals et al. Nature 2019 (AlphaStar); Brown & Sandholm Science 2017 (Libratus, poker CFR). The full picture of how to handle non-transitive games at scale.

Weekend 4 (model-based extensions). Schrittwieser et al. Nature 2020 (MuZero); Ye et al. NeurIPS 2021 (EfficientZero); Antonoglou et al. ICLR 2022 (Stochastic MuZero).

Codebase tour

If you want to read more code than you write, three open-source AlphaZero / MuZero implementations are worth knowing:

- **KataGo** (https://github.com/lightvector/KataGo): The strongest open-source Go engine; descended from AlphaGo Zero. The training-data pipeline is particularly well-engineered.
- **OpenSpiel** (https://github.com/deepmind/open_spiel): DeepMind's game-RL library; has clean implementations of CFR, PSRO, AlphaZero, and dozens of games.
- **MuZero-General** (https://github.com/werner-duvaud/muzero-general): A clean MuZero reproduction with a tiny code footprint; recommended starting point for understanding the MuZero codebase.

Open problems

- *Continuous-action AlphaZero*: MCTS with infinite action spaces. Progressive widening (Couetoux et al. 2011), neural MCTS over latent action distributions; still an open area.
- *Partial-information AlphaZero*: poker-style hidden information; CFR + neural nets is the state of the art (DeepStack, Libratus, Pluribus), but unifying with the AlphaZero recipe is still an active research direction.
- *Cooperative MARL with strategic agents*: most MARL benchmarks assume cooperative or zero-sum. Mixed-incentive cases (synthetic insurance brokers vs. platform; ATS recruiters vs. candidates) need mechanism-design awareness layered on top.

Sourced from `research/deeprl_core_sota.md` sections covering MARL, AlphaZero, and modern self-play.